The University of Hong Kong
School of Computing and Data Science

Detail Project Proposal for COMP7705

Agentic-Quant: A Multi-Agent Framework for Reasoning-Based Quantitative Analysis

Mentor: Prof. Wu, Chuan

Group Members

Full Name	Student ID	Email
Ying Tingkai	3036657615	tkying2025@connect.hku.hk
Wang Wenhan	3036656398	u3665639@connect.hku.hk
Cao Yujuncheng	3036654819	U3665481@connect.hku.hk
Wang Pengcheng	3036656427	u3665642@connect.hku.hk
Gao Ziteng	3036654259	U3665425@connect.hku.hk

March 2026

Abstract

Agentic-Quant is a multi-agent stock analysis framework built on LangGraph. Five AI agents — market analyst, news analyst, fundamentals analyst, risk analyst, and portfolio manager — work in a structured pipeline to produce trading recommendations from real-time market data. The system already supports data retrieval from Yahoo Finance and Google News, technical charting, historical backtesting, and a Streamlit-based web interface. This proposal describes five planned extensions: persistent context management, a human-in-the-loop approval mechanism, a simulated broker engine, multi-channel notification, and decision-flow visualization. Together, these additions aim to turn Agentic-Quant from a single-query advisory tool into a continuously running, auditable trading assistant.

Keywords: Multi-Agent System, LangGraph, LLM-driven Quantitative Analysis, Decision-Flow Visualization, Autonomous Trading, Human-in-the-Loop

1. Introduction

1.1. Background and Motivation

Large language models have introduced new ways to analyze financial markets. Unlike rule-based or statistical models, LLMs can read earnings reports, news articles, and analyst commentary, then produce structured reasoning in natural language. Recent studies show that LLM-based systems can generate stock signals with measurable alpha over market benchmarks [1].

However, a thorough investment decision involves multiple distinct tasks: reading price charts, tracking news, reviewing company fundamentals, assessing risk, and making a final call. Asking a single model to handle all of these creates a bottleneck — the model must switch between roles, hold excessive context, and cannot specialize. Multi-agent systems solve this by assigning each task to a dedicated agent [2], [3]. Role specialization and parallel execution lead to better results and more transparent reasoning.

Agentic-Quant adopts this approach using LangGraph, a graph-based workflow engine that supports parallel branching, conditional routing, and tool integration. The framework runs five agents in a structured pipeline and has produced working results. This proposal describes the current state and outlines five extensions toward a complete trading assistant.

1.2. Problem Statement

Most LLM-based financial tools operate as stateless, single-turn systems: the user submits a query, the model responds, and all context is lost. This creates three practical limitations:

1. **No memory.** The system cannot track past recommendations or evaluate their outcomes.
2. **No oversight.** High-risk decisions — such as large position changes when signals conflict — proceed without human review.
3. **No execution loop.** The system remains advisory only, with no mechanism to validate decisions against actual market outcomes.

1.3. Research Objectives

This project addresses the above gaps through five objectives:

1. Build a persistent context layer that retains agent reports, tool logs, and past decisions across sessions.

2. Implement a human-in-the-loop mechanism that routes high-risk or conflicting decisions to human review.
3. Develop a broker simulation engine that executes virtual trades and feeds results back to the agents.
4. Integrate messaging channels (WhatsApp, Telegram) for mobile monitoring and approval.
5. Create a decision-flow visualization that traces how each agent contributes to the final recommendation.

2. Literature Review

2.1. Multi-Agent Systems for Financial Analysis

Several recent systems apply multi-agent architectures to trading. TradingAgents [2] assigns roles — fundamental analyst, sentiment analyst, technical analyst — to separate LLM agents, with Bull and Bear researchers debating market conditions before a fund manager makes the final call. Experiments show improvements in cumulative return and Sharpe ratio over single-agent baselines.

FinCon [3], presented at NeurIPS 2024, uses a manager-analyst hierarchy modeled on real investment firms. It introduces *conceptual verbal reinforcement*: agents update beliefs from past outcomes and propagate them to relevant peers. A risk-control component critiques decisions at the episode level. The system generalizes across single-stock trading and portfolio management.

TradingGroup [4] deploys five agents (news sentiment, financial report, forecasting, style preference, and trading decision) with a self-reflection loop that distills past successes and failures. Backtests on five stock datasets outperform rule-based, ML, RL, and other LLM-based methods. FinMem [5] adds a layered memory module aligned with human cognitive structures, enabling agents to retain critical information beyond typical context windows.

These systems share a common design principle: decompose the analysis task by role, let each agent specialize, and aggregate results through a central decision-maker. Agentic-Quant follows the same pattern.

2.2. LLM-Driven Quantitative Trading

MarketSenseAI [1] uses GPT-4 with chain-of-thought prompting to analyze trends, news, fundamentals, and macroeconomic factors. Testing on S&P 100 stocks over 15 months yielded 10–30% excess alpha and up to 72% cumulative returns. The study shows that structured reasoning, not just raw prediction, is key to LLM effectiveness in finance.

BloombergGPT [6] demonstrates the value of domain-specific training: a 50-billion-parameter model trained on 363 billion tokens of financial data outperforms general-purpose models on financial NLP tasks. FinGPT [7] takes the opposite approach — open-source and lightweight, using low-rank adaptation to specialize general models at low cost. Both directions inform the model selection strategy for Agentic-Quant's agents.

A survey covering over fifty studies [8] identifies key challenges for production deployment: temporal data leakage in evaluation, hallucinated facts, limited data coverage, and high inference costs. These concerns directly shape Agentic-Quant's emphasis on data grounding, structured output validation, and cost-aware model selection.

2.3. Agent Orchestration and Decision Visualization

AGORA [9], presented at ACL 2025, provides a graph-based orchestration engine for LLM agents. Its evaluation reveals that simpler reasoning methods like chain-of-thought often match more complex

approaches at lower cost — a practical insight for agent design. FinAgent [10], from KDD 2024, is a multimodal agent combining numerical, textual, and visual data with tool augmentation and a dual-level reflection module, achieving over 36% improvement on profit metrics across six datasets.

On explainability, de la Rica Escudero et al. [11] apply SHAP and LIME to explain portfolio decisions made by deep RL agents in real time. Tatsat and Shater [12] go further with mechanistic interpretability, reverse-engineering LLM internals to understand financial reasoning. Wang et al. [13] show that modeling interactions *among* news items — rather than treating each independently — captures temporal dynamics that simple sentiment scoring misses. These findings motivate Agentic-Quant's decision-flow visualization: making the full reasoning chain from data to decision visible and auditable.

2.4. Human-in-the-Loop and Autonomous Trading

Alpha-GPT 2.0 [14], from HKUST and IDEA Research, introduces an iterative human-AI collaboration framework for quantitative investment. Rather than full automation, it lets human researchers guide AI in alpha mining while AI results inspire human insight. The paper argues that purely automated approaches face diminishing returns.

A comprehensive survey [15] covering 167 publications on reinforcement learning in quantitative finance maps RL concepts to investing language and evaluates multi-agent RL approaches for portfolio management. This body of work provides the baseline against which LLM-based systems like Agentic-Quant can be compared. A broader survey on LLM agents in finance [16] identifies coordination-aware multi-agent systems as an under-explored direction — precisely the gap Agentic-Quant aims to fill.

3. System Design and Current Implementation

3.1. Overall Architecture

Agentic-Quant uses LangGraph's `StateGraph` to orchestrate five agents. Figure 1 shows the pipeline. Three analyst agents run in parallel; the risk analyst and PM run sequentially after all upstream reports are available.

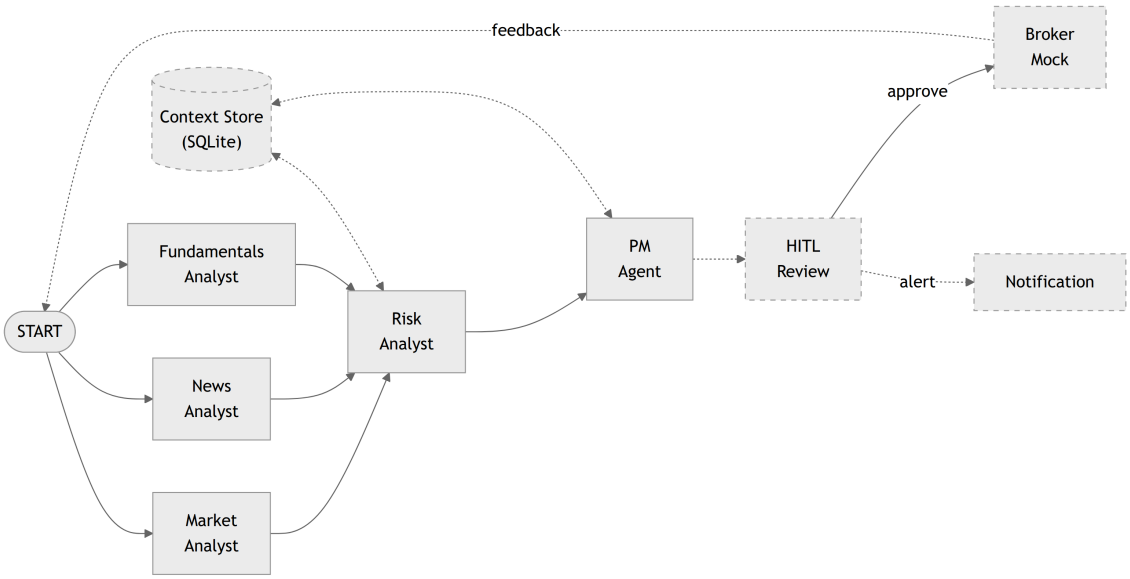

Figure 1: Target system architecture. Solid borders and arrows show existing components; dashed elements indicate planned enhancements.

The shared state holds the stock ticker, analysis date, current position, and per-agent message histories. Agents read from and write to this state but do not call each other directly. This keeps them loosely coupled and independently modifiable.

3.2. Agent Design

The five agents serve distinct roles:

- **Market Analyst** retrieves price data and technical indicators (SMA, Bollinger Bands, RSI, MACD) via Yahoo Finance. It identifies trends, support/resistance levels, and momentum signals.
- **News Analyst** fetches recent headlines from Google News, assesses sentiment and relevance, and highlights events that may affect near-term price action.
- **Fundamentals Analyst** pulls financial statements and valuation metrics from Yahoo Finance and AkShare. It evaluates profitability, growth, and valuation relative to sector peers.
- **Risk Analyst** reads the three upstream reports. It identifies conflicting signals, assesses overall risk exposure, and flags potential concerns.
- **Portfolio Manager (PM)** reads all four reports and outputs a structured decision via Pydantic schema: direction (bullish / bearish / neutral), target position, confidence, time horizon, and a reasoning summary.

Each agent runs as a LangGraph node backed by an LLM (currently OpenAI-compatible models). Agents that need external data access tools through LangGraph's `ToolNode` mechanism.

3.3. Data Pipeline

A unified `DataService` class provides three data sources:

- **Yahoo Finance** for real-time prices, technical indicators, and basic financials.
- **Google News** for news headlines and article links.
- **AkShare** for historical point-in-time fundamental data, with a focus on China A-shares.

A macro calendar module is in early development for economic event tracking. Retrieval failures trigger fallback logic; additional providers (e.g., Alpha Vantage) are planned as backup sources.

3.4. Web Interface

The Streamlit-based frontend offers three functions: stock analysis with configurable parameters, historical backtesting with performance charts (Plotly), and technical chart overlays with selectable indicators. Results can be exported as PDF reports.

4. Proposed Enhancements

4.1. Context Management and Persistent Memory

Currently all context lives in an in-memory Python dictionary and is lost after each run. The planned upgrade introduces a SQLite-based storage layer that persists agent reports, tool call logs with timestamps, PM decisions with reasoning, and per-ticker decision history.

A `ContextStore` abstraction will support swapping between in-memory (development) and SQLite (production) backends without changing application code. With persistent context, the system can replay any past analysis session, retrieve the last N decisions for a given ticker, and remain queryable after restart [5].

4.2. Human-in-the-Loop Approval Mechanism

In the current pipeline, the PM's decision is final. For a practical trading assistant, certain actions require human verification — especially large position changes, trades against conflicting signals, or decisions based on sparse data [14].

The HITL module will define trigger policies (e.g., position change above a threshold, analyst disagreement above a tolerance). Flagged decisions are routed to a human reviewer who can approve, reject, or modify parameters. All outcomes are logged and fed back for policy refinement.

4.3. Broker Mock Engine and Backtesting Loop

To close the gap between analysis and execution, a simulated broker engine will handle basic order types (market and limit), maintain virtual account balances and positions, and track profit/loss over time. The engine exposes a standard gateway interface modeled on real broker APIs (e.g., Moomoo OpenAPI), simplifying future migration to live trading [17].

This creates a full cycle: agents analyze, PM decides, broker executes, results feed back for the next round. The backtesting loop can also evaluate how well past decisions would have performed, providing a concrete measure of system quality.

4.4. Multi-Channel Notification

The system will integrate with WhatsApp and Telegram via their respective APIs. Planned notifications include: daily market summaries generated by the agents, real-time alerts when significant events are flagged, and HITL approval requests that reviewers can respond to directly from the messaging app.

4.5. Decision-Flow Visualization

Each analysis session produces a chain of intermediate outputs: data retrievals, agent reports, risk assessments, and a final decision. The visualization module will render this chain as an interactive flow diagram in the Streamlit interface, showing how each agent's analysis contributed to the final call. This serves as both an audit trail for users and a debugging tool for developers [11], [12].

5. Technical Stack

Category	**Technology**
Agent orchestration	LangGraph (StateGraph, ToolNode, MemorySaver)
LLM backend	OpenAI-compatible API
Data sources	Yahoo Finance, Google News, AkShare
Web interface	Streamlit, Plotly
Persistent storage	SQLite (planned)
Broker simulation	Custom engine with FastAPI gateway (planned)
Messaging	WhatsApp Business API, Telegram Bot API (planned)
Structured output	Pydantic
Package management	uv

Category	Technology
Language	Python 3.12

6. Milestones and Timeline

#	Task	Target Date	Hours
1	Data pipeline (Yahoo Finance, Google News, AkShare)	2026-02	80
2	Agent tools and LangGraph multi-agent workflow	2026-03	120
3	Streamlit web interface and basic backtesting	2026-03	100
Tasks 1–3 completed before proposal submission			
4	Context management and persistent storage (SQLite)	2026-04-14	150
5	Human-in-the-loop approval mechanism	2026-05-05	120
6	Broker mock engine and backtesting loop	2026-06-01	200
7	Decision-flow visualization	2026-06-16	150
8	Multi-channel notification and system integration	2026-07-06	160
9	Project webpage and end-to-end evaluation	2026-07-13	100
10	Final report and demo preparation	2026-07-17	120
		Total	**1300**

7. Deliverables

1. A working multi-agent stock analysis system with persistent context, HITL approval, and broker simulation.
2. A Streamlit-based web dashboard with decision-flow visualization.
3. Multi-channel notification integration (WhatsApp / Telegram).
4. A final report documenting system design, implementation, and evaluation results.
5. A demo video showing the end-to-end workflow.

References

[1] G. Fatouros, K. Metaxas, J. Soldatos, and D. Kyriazis, "Can Large Language Models beat wall street? Evaluating GPT-4's impact on financial decision-making with MarketSenseAI," *Neural Computing and Applications*, vol. 37, no. 30, pp. 24893–24918, Oct. 2025, doi: 10.1007/s00521-024-10613-4.

[2] Y. Xiao, E. Sun, D. Luo, and W. Wang, "TradingAgents: Multi-Agents LLM Financial Trading Framework." Accessed: Mar. 25, 2026. [Online]. Available: http://arxiv.org/abs/2412.20138

[3] Y. Yu *et al.*, "FinCon: A Synthesized LLM Multi-Agent System with Conceptual Verbal Reinforcement for Enhanced Financial Decision Making." Accessed: Mar. 25, 2026. [Online]. Available: http://arxiv.org/abs/2407.06567

[4] F. Tian, F. D. Salim, and H. Xue, "TradingGroup: A Multi-Agent Trading System with Self-Reflection and Data-Synthesis." Accessed: Mar. 25, 2026. [Online]. Available: http://arxiv.org/abs/2508.17565

[5] Y. Yu *et al.*, "FinMem: A Performance-Enhanced LLM Trading Agent with Layered Memory and Character Design." Accessed: Mar. 25, 2026. [Online]. Available: http://arxiv.org/abs/2311.13743

[6] S. Wu *et al.*, "BloombergGPT: A Large Language Model for Finance." Accessed: Mar. 25, 2026. [Online]. Available: http://arxiv.org/abs/2303.17564

[7] H. Yang, X.-Y. Liu, and C. D. Wang, "FinGPT: Open-Source Financial Large Language Models." Accessed: Mar. 25, 2026. [Online]. Available: http://arxiv.org/abs/2306.06031

[8] W. Fu, "The New Quant: A Survey of Large Language Models in Financial Prediction and Trading." Accessed: Mar. 25, 2026. [Online]. Available: http://arxiv.org/abs/2510.05533

[9] Q. Zhang *et al.*, "Unifying Language Agent Algorithms with Graph-based Orchestration Engine for Reproducible Agent Research," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, Vienna, Austria: Association for Computational Linguistics, 2025, pp. 107–117. doi: 10.18653/v1/2025.acl-demo.11.

[10] W. Zhang *et al.*, "A Multimodal Foundation Agent for Financial Trading: Tool-Augmented, Diversified, and Generalist," in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, Barcelona Spain: ACM, Aug. 2024, pp. 4314–4325. doi: 10.1145/3637528.3671801.

[11] A. de-la-Rica-Escudero, E. C. Garrido-Merchán, and M. Coronado-Vaca, "Explainable post hoc portfolio management financial policy of a Deep Reinforcement Learning agent," *PLOS ONE*, vol. 20, no. 1, p. e315528, Jan. 2025, doi: 10.1371/journal.pone.0315528.

[12] H. Tatsat and A. Shater, "Beyond the Black Box: Interpretability of LLMs in Finance." Accessed: Mar. 25, 2026. [Online]. Available: http://arxiv.org/abs/2505.24650

[13] M. Wang, S. B. Cohen, and T. Ma, "Modeling News Interactions and Influence for Financial Market Prediction," in *Findings of the Association for Computational Linguistics: EMNLP 2024*, Miami, Florida, USA: Association for Computational Linguistics, 2024, pp. 3302–3314. doi: 10.18653/v1/2024.findings-emnlp.189.

[14] H. Yuan, S. Wang, and J. Guo, "Alpha-GPT 2.0: Human-in-the-Loop AI for Quantitative Investment." Accessed: Mar. 25, 2026. [Online]. Available: http://arxiv.org/abs/2402.09746

[15] N. Pippas, E. A. Ludvig, and C. Turkay, "The Evolution of Reinforcement Learning in Quantitative Finance: A Survey," *ACM Computing Surveys*, vol. 57, no. 11, pp. 1–51, Nov. 2025, doi: 10.1145/3733714.

[16] Y. Dong *et al.*, "Large Language Model Agents in Finance: A Survey Bridging Research, Practice, and Real-World Deployment," in *Findings of the Association for Computational Linguistics: EMNLP 2025*, Suzhou, China: Association for Computational Linguistics, 2025, pp. 17889–17907. doi: 10.18653/v1/2025.findings-emnlp.972.

[17] H. Yang *et al.*, "FinRobot: An Open-Source AI Agent Platform for Financial Applications using Large Language Models." Accessed: Mar. 25, 2026. [Online]. Available: http://arxiv.org/abs/2405.14767